

JOIN példák

Táblák nem megfelelő összekötése

Az ID mezőt a lehető legritkábban kötjük össze a másik tábla ID mezőjével!

DESCARTES szorzat

```
SELECT *
FROM cikkek, felhasznalok
WHERE cikkek.id = felhasznalok.id;
```

INNER JOIN

```
SELECT *
FROM cikkek
    INNER JOIN felhasznalok
        ON cikkek.id = felhasznalok.id;
```

id	felhasznalo_id	cim	szoveg	id	nev
1	1	Első cikk	NULL	1	Norbi
2	2	Új motorom	NULL	2	Bea
3	3	Hogyan lettem videós	NULL	3	Helga

- Hiányzik a 4-es cikk: "Új nap kezdődik"
- Bea nem írt cikket, mégis úgy néz ki, mintha ő írta volna a 2-es cikket.
- A 2-es cikket nem Bea, hanem Helga írta

Helyes megoldás DESCARTES szorzattal

```
SELECT *
FROM cikkek, felhasznalok
WHERE cikkek.felhasznalo_id = felhasznalok.id;
```

Helyes megoldás INNER JOIN-nal

```
SELECT *
FROM cikkek
    INNER JOIN felhasznalok
        ON cikkek.felhasznalo_id = felhasznalok.id;
```

Eredmény

id	felhasznalo_id	cim	szoveg	id	nev
1	1	Első cikk	NULL	1	Norbi
2	2	Új motorom	NULL	2	Bea
3	3	Hogyan lettem videós	NULL	3	Helga
4	1	Új nap kezdődik	NULL	1	Norbi

Számít a sorrend?

A kérdés az, hogy hol számít! A **FROM** után számít.

```
SELECT *
FROM cikkek
      INNER JOIN felhasznalok
            ON cikkek.felhasznalo_id = felhasznalok.id;
```

id	felhasznalo_id	cim	szoveg	id	nev
1	1	Első cikk	NULL	1	Norbi
2	2	Új motorom	NULL	2	Bea
3	3	Hogyan lettem videós	NULL	3	Helga
4	1	Új nap kezdődik	NULL	1	Norbi

A fenti példában a bal oldalon a `cikkek` tábla adatai jelennek meg, és a `felhasznalok` tábla adatai kerülnek a jobb oldalra.

```
SELECT *
FROM felhasznalok
      INNER JOIN cikkek
            ON cikkek.felhasznalo_id = felhasznalok.id;
```

id	nev	id	felhasznalo_id	cim	szoveg
1	Norbi	1	1	Első cikk	NULL
2	Bea	2	2	Új motorom	NULL
3	Helga	3	3	Hogyan lettem videós	NULL
1	Norbi	4	1	Új nap kezdődik	NULL

Míg ebben a példában a bal oldalon a `felhasznalok` tábla adatai jelennek meg, és a `cikkek` tábla adatai kerülnek a jobb oldalra.

Mi a helyzet a kapcsolódási feltétellel?

```
SELECT *
FROM cikkek
      INNER JOIN felhasznalok
            ON cikkek.felhasznalo_id = felhasznalok.id;
```

```
SELECT *
FROM cikkek
      INNER JOIN felhasznalok
            ON felhasznalok.id = cikkek.felhasznalo_id;
```

Mivel az `ON` záradékba logikai feltétel kerül, így az egyenlőség két oldala felcserélhető, ugyanazt eredményezi.

INNER JOIN vs DESCARTES és WHERE

A feladatok többféleképpen is megoldhatóak. A `JOIN` abból a szempontból előnyösebb, hogy az összekapcsolás feltétele és a szűrés feltétele nem keveredik.

Jelenítse meg Norbi és Helga cikkeit!

```
SELECT *
FROM cikkek, felhasznalok
WHERE cikkek.felhasznalo_id = felhasznalok.id
      AND felhasznalok.nev = 'Norbi' OR felhasznalok.nev = 'Helga';
```

id	felhasznalo_id	cim	szoveg	id	nev
1	1	Első cikk	NULL	3	Helga
1	1	Első cikk	NULL	1	Norbi
2	2	Új motorom	NULL	3	Helga
3	3	Hogyan lettem videós	NULL	3	Helga
4	1	Új nap kezdődik	NULL	3	Helga
4	1	Új nap kezdődik	NULL	1	Norbi

- Összesen 4 cikk van, de több sort kaptunk
- Helga2 cikket írt valójában, de mintha 4 lenne az eredmény szerint
- A hiba oka a hibás zárójelezés miatt az `AND`-nek magasabb a prioritása, így az összeköttetést nem figyeli Helga nevénél.

```
SELECT *
FROM cikkek, felhasznalok
WHERE (cikkek.felhasznalo_id = felhasznalok.id)
      AND (felhasznalok.nev = 'Norbi' OR felhasznalok.nev = 'Helga');
```

- Az összeköttetés feltétele és a szűrés is külön zárójelet kapott.

```
SELECT *
FROM cikkek
      INNER JOIN felhasznalok
              ON felhasznalok.id = cikkek.felhasznalo_id
WHERE
      felhasznalok.nev = 'Norbi' OR felhasznalok.nev = 'Helga';
```

- `JOIN` esetében ez így nem okoz gondot.

```
SELECT *
FROM cikkek, felhasznalok
WHERE (cikkek.felhasznalo_id = felhasznalok.id)
      AND (felhasznalok.nev IN ('Norbi', 'Helga'));
```

- Erre a konkrét példára az `IN()` is megoldás lehet, de ez nem minden esetben mondható el.

Mi kerülhet az ON záradékba

```
SELECT *
FROM cikkek
      INNER JOIN felhasznalok
```

```
ON felhasznalok.id = cikkek.felhasznalo_id
AND felhasznalok.nev = 'Norbi';
```

- Az **ON** záradékba logikai feltételnek kell kerülnie. **Tipikusan ez a helye a táblák összekapcsolásához szükséges feltételnek.**
- Akár szűrési feltétel is szerepelhet itt, ilyenkor a megfelelő zárójelzésre oda kell figyelni.
- Bizonyos esetekben gyorsabbá teheti a lekérdezést.

id	felhasznalo_id	cim	szoveg	id	nev
1	1	Első cikk	NULL	1	Norbi
4	1	Új nap kezdődik	NULL	1	Norbi

Mindig pontosan a szükséges táblák legyenek összekötve

A feladatoknál oda kell figyelni, hogy mindig a megfelelő táblák legyenek összekapcsolva!

Példa: Hány szerzőt tartalmaz az adatbázis?

```
SELECT COUNT(*) AS `szerzok_szama`
FROM cikkek
    INNER JOIN felhasznalok
        ON felhasznalok.id = cikkek.felhasznalo_id;
```

```
SELECT COUNT(`felhasznalo_id`) AS `szerzok_szama`
FROM cikkek
    INNER JOIN felhasznalok
        ON felhasznalok.id = cikkek.felhasznalo_id;
```

A fenti kódok 4 felhasználót mondanak, pedig 3 van. Itt elegendő lett volna csak a **felhasznalok** tábla.

```
SELECT COUNT(distinct `felhasznalo_id`) AS `szerzok_szama`
FROM cikkek
    INNER JOIN felhasznalok
        ON felhasznalok.id = cikkek.felhasznalo_id;
```

- Az eredmény 2 lesz. Bea kiesett
- Nem sokkal jobb megoldás, ugyanis a fölösleges összeköttetést el kellett végeznie az adatbázisnak, továbbá a szintén köljdtészes egyedi értékek kiválogatását is el kellett végeznie.

Összekötő táblák

Amennyiben két tábla között egy úgynevezett összekötő tábla áll, úgy az nagy eséllyel nem hagyható ki a lekérdezésből.

Az is előfordulhat, hogy az összekötő tábla már tartalmaz számunkra szükséges információt, például darabszám, így 3 helyett elég 2 táblát összekapcsolni, ahol az egyik az összekötő tábla.